

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH



BÁO CÁO:
KHẢ NĂNG HỌC CHÍNH XÁC
THUẬT TOÁN CỦA MẠNG NƠ-RON

*Bài báo gốc của Back de Luca et al. (2025):
Learning to Add, Multiply, and Execute
Algorithmic Instructions Exactly with Neural Networks*

Môn học: Nhập môn Học máy

Lớp: 23KHMT3

Nhóm: 11

Giảng viên: PGS.TS. Lê Hoàng Thái

ThS. Lê Nhật Nam

ThS. Trần Trung Kiên

Sinh viên: Trần Nguyễn Khải Luân – 23127006

Nguyễn Bảo Duy – 23127179

Trương Quốc Cường – 23127333

Nguyễn Thanh Tiến – 23127539

Thông tin chung

Bảng công việc

Họ tên	Công việc	Hoàn thành
Nguyễn Thanh Tiến	Tiến hành thực nghiệm và trực quan hóa kết quả Chương 4	100%
	Làm slide phần Thực nghiệm và Kết luận	
Trần Nguyễn Khải Luân	Viết Chương 1: Tổng quan (Giới thiệu, Các nghiên cứu liên quan, Động lực nghiên cứu)	100%
	Viết Chương 5: Kết luận	
	Làm slide phần Tổng quan	
Trương Quốc Cường	Viết Chương 2: Cơ sở lý thuyết (NTK, Giới hạn chiều rộng vô hạn)	100%
	Viết Chương 4: Thực nghiệm và Phân tích	
	Làm slide phần Cơ sở lý thuyết và Khớp mẫu	
Nguyễn Bảo Duy	Viết Chương 3: Phương pháp đề xuất (Khớp mẫu, Trực giao hóa, Định lý NTK)	100%
	Tổng hợp tài liệu tham khảo và bảng thuật ngữ	
	Làm slide phần Khả năng học chính xác	

Mục lục

Bảng thuật ngữ và từ viết tắt	1
1 GIỚI THIỆU VÀ ĐỘNG LỰC	3
1.1 Giới thiệu	3
1.2 Động lực nghiên cứu	3
1.3 Câu hỏi nghiên cứu	4
1.4 Đóng góp của báo cáo	4
2 NGHIÊN CỨU LIÊN QUAN	6
2.1 Các nghiên cứu liên quan	6
2.2 Cơ sở lý thuyết	6
2.2.1 Ký hiệu và các khái niệm cơ bản	6
2.2.2 Neural Tangent Kernel (NTK)	7
2.2.3 Giới hạn chiều rộng vô hạn và Động lực học	8
2.2.4 Kiến trúc mạng nơ-ron hai lớp	10
3 PHƯƠNG PHÁP ĐỀ XUẤT	11
3.1 Điều kiện về khả năng học chính xác	11
3.1.1 Giả thuyết 5.1: Kiểm soát tương quan không mong muốn	11
3.1.2 Vai trò của hàm làm tròn	12
3.2 Kỹ thuật Khớp mẫu	12
3.2.1 Nguyên lý phân rã thuật toán	12
3.2.2 Cơ chế thực thi lặp	13
3.3 Cấu trúc dữ liệu và Trục giao hóa	13
3.3.1 Mô hình Bag of Rule	13
3.3.2 Mã hóa phân vùng khối	14
3.3.3 Trục giao hóa và Ma trận NTK	14
3.4 Định lý về bộ dự báo NTK	14
3.4.1 Định lý 5.1: Bảo toàn thông tin dấu	14
3.4.2 Ý nghĩa của Định lý	15
3.5 Kiểm soát tương quan không mong muốn	15
3.5.1 Giả thuyết 5.1: Điều kiện về tương quan	15
3.5.2 Phân tích trường hợp xấu nhất	16
3.5.3 Kết quả cho các thuật toán cụ thể	16
3.6 Triển khai thuật toán cụ thể	16
3.6.1 Phép hoán vị	16
3.6.2 Phép cộng	17
3.6.3 Phép nhân	17
3.6.4 Chỉ thị SBN (Subtract and Branch if Negative)	17

3.7	Độ phức tạp tập hợp	18
3.7.1	Tại sao cần kỹ thuật kết hợp nhiều mô hình?	18
3.7.2	Công thức tính số lượng mô hình tối thiểu	18
3.7.3	Độ phức tạp theo kích thước bài toán	19
4	PHÂN TÍCH THỰC NGHIỆM	20
4.1	Thiết lập thực nghiệm	20
4.1.1	Kiến trúc mạng	20
4.1.2	Tham số khởi tạo	20
4.1.3	Cấu hình huấn luyện	20
4.1.4	Đối tượng kiểm thử	21
4.2	Xác minh lý thuyết	21
4.2.1	Phương pháp xác minh	21
4.2.2	Kết quả trên phép cộng và nhân	21
4.2.3	So sánh phương sai và kỳ vọng	22
4.3	Tổng hợp và Đánh giá	23
5	MÔ PHỎNG THỰC NGHIỆM	24
5.1	Lựa chọn bài toán	24
5.2	Bài toán Hoán vị	24
5.3	Độ chính xác theo kích thước của tập hợp các mô hình	25
5.4	Phân tích kết quả	26
5.5	Thực nghiệm với bài toán Cộng nhị phân	26
5.6	Kết luận từ thực nghiệm	27
5.7	Ý nghĩa thực tiễn	28
6	PHÂN TÍCH PHÊ BÌNH	29
7	KẾT LUẬN	30
7.1	Tổng kết nghiên cứu	30
7.2	Ý nghĩa của nghiên cứu	30
7.3	Hướng phát triển tương lai	31
	Tài liệu tham khảo	32

Danh sách bảng

1	Danh sách các từ viết tắt	1
2	Bảng thuật ngữ Anh–Việt	2

Danh sách hình vẽ

4.1	Tỷ lệ σ^2/μ^2 theo kích thước đầu vào k' . Đường nét đứt biểu diễn dự đoán lý thuyết với tăng trưởng tuyến tính. Các điểm thực nghiệm phù hợp chặt chẽ với dự đoán này.	22
5.1	Thực nghiệm 1 (trái) trình bày số mô hình cần thiết để đạt độ chính xác 90% theo kích thước đầu vào k , so sánh với ngưỡng lý thuyết từ công thức tính số lượng mô hình tối thiểu. Thực nghiệm 2 (phải) trình bày độ chính xác theo số mô hình với các kích thước đầu vào k khác nhau.	25
5.2	Kết quả nhóm thực nghiệm lại các thí nghiệm trong bài báo.	26

Bảng thuật ngữ và từ viết tắt

Bảng 1: Danh sách các từ viết tắt

Viết tắt	Ý nghĩa
NTK	Neural Tangent Kernel (hạt nhân tiếp tuyến nơ-ron)
MSE	Mean Squared Error (sai số bình phương trung bình)
RNN	Recurrent Neural Network (mạng nơ-ron hồi quy)
GNN	Graph Neural Network (mạng nơ-ron đồ thị)
LSB	Least Significant Bit (bit phải cùng)
SBN	Subtract and Branch if Negative (lệnh trừ và nếu âm thì nhảy)
i.i.d.	independent and identically distributed (độc lập và phân phối giống nhau)
GPU	Graphics Processing Unit (bộ xử lý đồ họa)
TPU	Tensor Processing Unit (bộ xử lý tensor)
ReLU	Rectified Linear Unit (hàm kích hoạt ReLU)
ODE	Ordinary Differential Equation (phương trình vi phân thường)

Bảng 2: Bảng thuật ngữ Anh–Việt

Thuật ngữ tiếng Anh	Cách dịch thống nhất
loss function	hàm mất mát
bias	hệ số chệch
template	mẫu
block	khối
template matching	khớp mẫu
transfer learning	học chuyển giao
unwanted correlation	tương quan không mong muốn
ensemble complexity	độ phức tạp tập hợp
training experiments	thực nghiệm huấn luyện
kernel regression	hồi quy hạt nhân
Assumption	giả thuyết
bitwise disjunction	phép tuyển theo bit
shift-and-add	thuật toán dịch và cộng
multiplier / multiplicand	số nhân / số bị nhân
exact learnability	khả năng học chính xác
closed-form	dạng tường minh
rank-1 update/noise	cập nhật/nhiều hạng 1
lazy training	huấn luyện lười

Chương 1

GIỚI THIỆU VÀ ĐỘNG LỰC

1.1 Giới thiệu

Mạng nơ-ron đã chứng minh khả năng xấp xỉ vượt trội đối với các hàm liên tục và trơn, đặc biệt trong các bài toán nhận dạng mẫu, xử lý ngôn ngữ tự nhiên và thị giác máy tính. Tuy nhiên, khi đối mặt với các phép toán rời rạc như phép cộng nhị phân, phép nhân số học hay thực thi các chỉ thị thuật toán, mạng nơ-ron truyền thống thường gặp khó khăn đáng kể. Sự đối lập này phản ánh thách thức cơ bản: trong khi mạng nơ-ron được tối ưu hóa cho việc xấp xỉ hàm số trong không gian liên tục, các thuật toán thực tế yêu cầu sự chính xác tuyệt đối ở cấp độ bit.

Nghiên cứu đã đề xuất một hướng tiếp cận đột phá để giải quyết vấn đề này thông qua khung lý thuyết NTK. Bằng cách phân tích mạng nơ-ron hai lớp trong giới hạn chiều rộng vô hạn, các tác giả chứng minh rằng mạng nơ-ron có thể được huấn luyện để thực thi chính xác các chỉ thị thuật toán được mã hóa ở dạng nhị phân. Đây là kết quả có ý nghĩa lý thuyết quan trọng, mở ra khả năng sử dụng mạng nơ-ron như một công cụ tính toán phổ quát thay vì chỉ giới hạn ở vai trò xấp xỉ hàm số.

Mục tiêu cốt lõi của nghiên cứu là chứng minh một cách chặt chẽ về mặt toán học rằng mạng nơ-ron, khi được huấn luyện thông qua gradient descent với cơ chế khớp mẫu, có khả năng học và thực thi chính xác các chỉ thị thuật toán mà không cần xấp xỉ.

1.2 Động lực nghiên cứu

Một trong những thách thức cốt lõi khi huấn luyện mạng nơ-ron thực hiện các phép toán rời rạc là sự khó khăn trong việc hội tụ gradient descent đối với các hàm không liên tục. Gradient descent hoạt động dựa trên việc tính toán đạo hàm của hàm mất mát theo các tham số mạng, nhưng đối với các hàm bước nhảy hay các phép toán nhị phân, đạo hàm hầu như bằng không hoặc không xác định tại hầu hết các điểm. Điều này dẫn đến hiện tượng gradient biến mất hoặc gradient không cung cấp thông tin hữu ích cho việc cập nhật tham số.

Các bộ dữ liệu tiêu chuẩn cho bài toán số học thường không được thiết kế để giải quyết vấn đề này một cách có hệ thống. Hệ quả là mạng nơ-ron được huấn luyện trên các bộ dữ liệu như vậy có thể đạt độ chính xác cao trên tập huấn luyện nhưng thất bại khi khái quát hóa sang các ví dụ mới, đặc biệt khi kích thước đầu vào tăng lên.

Động lực chính của nghiên cứu này xuất phát từ nhu cầu tìm kiếm một cơ chế huấn luyện có thể đảm bảo mạng nơ-ron đạt được độ chính xác tuyệt đối thay vì chỉ xấp xỉ.

Thay vì cố gắng xấp xỉ một hàm rời rạc bằng một hàm liên tục, nghiên cứu này đề xuất mã hóa dữ liệu và thuật toán theo cách cho phép mạng nơ-ron nhận dạng và áp dụng các quy tắc cấp độ bit một cách chính xác thông qua cơ chế khớp mẫu. Cách tiếp cận này biến bài toán từ xấp xỉ hàm thành bài toán nhận dạng mẫu, nơi mạng nơ-ron có thể phát huy thế mạnh vốn có.

1.3 Câu hỏi nghiên cứu

Nghiên cứu đặt ra một câu hỏi tổng quát, gồm **ba phần** tương ứng với ba khía cạnh chính:

- **Về mặt lý thuyết:** Liệu có thể chứng minh rằng một mạng nơ-ron học được cơ chế khớp mẫu để thực thi **chính xác** các chỉ thị thuật toán, dưới các giả định như NTK và mô hình nhiễu?
- **Về hiệu quả dữ liệu:** Lượng dữ liệu huấn luyện tối thiểu cần thiết (độ phức tạp mẫu) là bao nhiêu?
- **Về kết nối với thực nghiệm:** Trong thực tế khi mạng có độ rộng hữu hạn và dữ liệu hữu hạn, các bảo đảm/dự đoán từ lý thuyết được phản ánh như thế nào?

Từ đó, báo cáo cụ thể hóa câu hỏi nghiên cứu thành **ba phần**:

- **Phần 1 (Lý thuyết/NTK).** Trong khuôn khổ NTK ở giới hạn chiều rộng vô hạn, liệu hạ gradient có thể hội tụ đến nghiệm thực thi **đúng** thuật toán (ở mức bit) hay không?
- **Phần 2 (Thực nghiệm/mạng hữu hạn).** Khi rời khỏi giả định chiều rộng vô hạn (mạng hữu hạn và dữ liệu hữu hạn), mô hình còn có thể học và khái quát hóa việc thực thi chỉ thị thuật toán ở mức độ nào? Mức suy giảm so với dự đoán/bảo đảm lý thuyết thể hiện ra sao theo độ dài bit, theo seed/khởi tạo, và theo quy mô dữ liệu?
- **Phần 3 (Độ phức tạp mẫu).** Lượng dữ liệu huấn luyện tối thiểu phụ thuộc như thế nào vào kích thước đầu vào?

1.4 Đóng góp của báo cáo

Nghiên cứu của đã mang lại 5 đóng góp chính về mặt lý thuyết và phương pháp luận:

- **Biểu diễn thuật toán dưới dạng các mẫu:** Mỗi quy tắc cấp độ bit của thuật toán được mã hóa thành một vector mẫu riêng biệt, cho phép cô lập các quy tắc logic và tạo ra cấu trúc dữ liệu phù hợp cho việc áp dụng cơ chế khớp mẫu.
- **Chứng minh lý thuyết dựa trên NTK:** Nghiên cứu cung cấp chứng minh chặt chẽ rằng mạng nơ-ron hai lớp trong giới hạn chiều rộng vô hạn có thể học chính xác: phép hoán vị, phép cộng nhị phân, phép nhân nhị phân, và chỉ thị SBN.
- **Hiệu quả về dữ liệu:** Phương pháp chỉ cần số lượng ví dụ huấn luyện ở quy mô logarit thay vì tuyến tính hoặc đa thức, cho phép huấn luyện hiệu quả ngay cả với các bài toán có kích thước đầu vào lớn.

- **Tính phổ quát:** Vì chỉ thị SBN mang tính đầy đủ Turing, việc chứng minh mạng nơ-ron có thể học chính xác SBN đồng nghĩa với việc mạng nơ-ron có thể, về nguyên tắc, học thực thi bất kỳ thuật toán nào.
- **Góc nhìn giáo dục:** Báo cáo này tổng hợp và trình bày lại các kết quả từ nghiên cứu gốc, giúp người đọc tiếp cận các khái niệm phức tạp như NTK, chiều rộng vô hạn, và cơ chế khớp mẫu một cách có hệ thống.

Các đóng góp này không chỉ có giá trị về mặt lý thuyết thuần túy mà còn mở ra hướng phát triển mới cho việc thiết kế và huấn luyện mạng nơ-ron trong các ứng dụng yêu cầu độ chính xác tuyệt đối, như xác minh phần cứng, mã hóa và giải mã, và các hệ thống nhúng quan trọng.

Chương 2

NGHIÊN CỨU LIÊN QUAN

2.1 Các nghiên cứu liên quan

Khả năng tính toán của mạng nơ-ron đã được nghiên cứu rộng rãi trong nhiều thập kỷ qua. Các nghiên cứu tiên phong về RNN đã chứng minh rằng RNN có thể mô phỏng máy Turing về mặt lý thuyết [26], nghĩa là chúng có khả năng biểu diễn bất kỳ hàm tính toán được nào. Gần đây hơn, kiến trúc Transformer cũng được chứng minh có khả năng biểu diễn tương đương [23], củng cố thêm quan điểm về tính phổ quát của mạng nơ-ron.

Tuy nhiên, điều quan trọng cần phân biệt là tính khả thi về mặt biểu diễn hoàn toàn khác với khả năng học được chính xác thông qua gradient descent. Một mạng nơ-ron có thể có khả năng biểu diễn một hàm phức tạp, nhưng điều đó không đảm bảo rằng thuật toán tối ưu hóa như gradient descent có thể tìm được tập tham số phù hợp để thực hiện hàm đó từ dữ liệu huấn luyện hữu hạn. Đây chính là khoảng cách then chốt mà các nghiên cứu trước đó chưa giải quyết triệt để.

Nhiều kiến trúc chuyên biệt đã được đề xuất để thu hẹp khoảng cách này. Neural GPU mở rộng khả năng của mạng tích chập để xử lý các phép toán số học song song. NALU được thiết kế đặc biệt để học các phép toán số học như cộng, trừ, nhân và chia. Mặc dù các kiến trúc này đạt được kết quả thực nghiệm tốt trong một số trường hợp, chúng thiếu nền tảng lý thuyết vững chắc về việc tại sao và khi nào quá trình học hội tụ đến giải pháp chính xác.

Nghiên cứu đã cho thấy sự khác biệt cơ bản ở chỗ chứng minh lý thuyết chặt chẽ dựa trên khung NTK, đảm bảo rằng mạng nơ-ron không chỉ có khả năng biểu diễn mà thực sự có thể học được các phép toán chính xác thông qua gradient descent với lượng dữ liệu huấn luyện có thể kiểm soát được.

2.2 Cơ sở lý thuyết

2.2.1 Ký hiệu và các khái niệm cơ bản

Trước khi đi vào phân tích chi tiết khung lý thuyết NTK, các ký hiệu toán học cơ bản được sử dụng xuyên suốt báo cáo này là:

1. Vector và ma trận:

- Vector cột n chiều: $\mathbf{x} \in \mathbb{R}^n$

- Phần tử thứ i của vector: x_i hoặc $[\mathbf{x}]_i$
- Ma trận đơn vị kích thước $n \times n$: I_n
- Tích vô hướng: $\langle \mathbf{x}, \mathbf{y} \rangle = \mathbf{x}^\top \mathbf{y} = \sum_{i=1}^n x_i y_i$

2. **Chuẩn Euclidean:** Chuẩn Euclidean (hay chuẩn L^2) của một vector $\mathbf{x}$ được định nghĩa là:

$$\|\mathbf{x}\| = \sqrt{\sum_{i=1}^n x_i^2} = \sqrt{\langle \mathbf{x}, \mathbf{x} \rangle} \quad (2.1)$$

3. **Tập hợp chỉ số:** Ký hiệu $[n]$ biểu thị tập hợp các số nguyên dương từ 1 đến n :

$$[n] = \{1, 2, \dots, n\} \quad (2.2)$$

4. **Bài toán hồi quy và hàm loss:** Xét một mạng nơ-ron với tham số θ thực hiện ánh xạ đầu vào $\mathbf{x}$ sang đầu ra $F(\mathbf{x}; \theta)$. Cho tập dữ liệu huấn luyện $\{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^N$ với N mẫu, mục tiêu huấn luyện là tối thiểu hóa hàm loss Mean Squared Error (MSE):

$$\mathcal{L}(\theta) = \frac{1}{2} \|F(\mathbf{x}; \theta) - \mathbf{y}\|^2 = \frac{1}{2} \sum_{i=1}^N \|F(\mathbf{x}_i; \theta) - \mathbf{y}_i\|^2 \quad (2.3)$$

Trong đó $\mathbf{y}$ biểu thị vector chứa tất cả các nhãn đầu ra mục tiêu.

• Tại sao chọn MSE?

- Đạo hàm $\frac{\partial \mathcal{L}}{\partial F} = F - \mathbf{y}$ có dạng tuyến tính, cho phép gradient descent hội tụ ổn định.
- Trong chế độ NTK, MSE loss dẫn đến nghiệm dạng tường minh theo hồi quy hạt nhân, tạo nền tảng cho phân tích lý thuyết chính xác.

$\Rightarrow$ Nhờ đó, ta có thể chứng minh rằng dấu của đầu ra dự đoán luôn khớp với nhãn mục tiêu (tức là nếu $y = +1$ thì $\hat{y} > 0$, và ngược lại). Điều này đảm bảo rằng sau khi làm tròn, kết quả sẽ chính xác tuyệt đối - đây là nền tảng cho khái niệm *Khả năng học chính xác* được trình bày trong Định lý 5.1 (Chương 3).

Lệnh **SBN** có dạng "*trừ và nếu kết quả âm thì nháy*", nên thỏa cả cập nhật trạng thái, điều kiện, và điều khiển luồng; do đó có thể đạt tính đầy đủ Turing khi được lặp lại/ghép chuỗi phù hợp [25, 24].

Tác giả đưa SBN vào để nhấn mạnh rằng khung khớp mẫu và phân tích NTK không chỉ nhắm tới các phép toán số học cụ thể, mà còn đóng vai trò **nền tảng lý thuyết** hướng tới việc học và thực thi **mọi thuật toán** [1].

2.2.2 Neural Tangent Kernel (NTK)

NTK là một công cụ lý thuyết mạnh mẽ để phân tích động lực học huấn luyện của mạng nơ-ron [2]. Ý tưởng cốt lõi là mô tả sự thay đổi của đầu ra mạng theo thời gian huấn luyện thông qua một kernel cố định.

Ma trận Jacobian

Xét mạng nơ-ron với đầu ra $F(\mathbf{x}; \theta) \in \mathbb{R}^k$ phụ thuộc vào vector tham số $\theta \in \mathbb{R}^p$. Ma trận Jacobian của đầu ra theo tham số được định nghĩa là:

$$J(\theta) = \frac{\partial F(\mathbf{x}; \theta)}{\partial \theta} \in \mathbb{R}^{Nk \times p} \quad (2.4)$$

trong đó N là số lượng mẫu dữ liệu, k là chiều của đầu ra, và p là số lượng tham số. Mỗi hàng của ma trận Jacobian biểu thị gradient của một thành phần đầu ra theo toàn bộ tham số.

Định nghĩa NTK thực nghiệm

Neural Tangent Kernel thực nghiệm được định nghĩa thông qua tích của ma trận Jacobian với chuyển vị của nó:

$$K(\theta) = J(\theta)J(\theta)^\top \in \mathbb{R}^{Nk \times Nk} \quad (2.5)$$

Phần tử (i, j) của ma trận kernel này đo lường mức độ tương quan giữa gradient của đầu ra thứ i và đầu ra thứ j trong không gian tham số. Cụ thể hơn, với hai điểm dữ liệu $\mathbf{x}$ và $\mathbf{x}'$, giá trị kernel được tính như sau:

$$K(\mathbf{x}, \mathbf{x}'; \theta) = \left\langle \frac{\partial F(\mathbf{x}; \theta)}{\partial \theta}, \frac{\partial F(\mathbf{x}'; \theta)}{\partial \theta} \right\rangle \quad (2.6)$$

Ý nghĩa vật lý của NTK

NTK cho phép hiểu rõ cách mạng nơ-ron học từ dữ liệu. Khi huấn luyện bằng gradient descent, sự thay đổi của đầu ra mạng tại một điểm $\mathbf{x}$ không chỉ phụ thuộc vào gradient tại điểm đó mà còn phụ thuộc vào gradient tại tất cả các điểm dữ liệu khác thông qua kernel. Đây chính là cơ chế *học chuyển giao* nội tại trong mạng nơ-ron: thông tin học được từ một mẫu có thể ảnh hưởng đến dự đoán tại các mẫu khác.

2.2.3 Giới hạn chiều rộng vô hạn và Động lực học

Một trong những đóng góp quan trọng nhất của lý thuyết NTK là khám phá hành vi của mạng nơ-ron khi chiều rộng lớp ẩn tiến tới vô hạn. Trong giới hạn này, việc huấn luyện mạng nơ-ron trở nên đơn giản hóa đáng kể và có thể được phân tích chính xác.

Chế độ huấn luyện lười

Khi chiều rộng lớp ẩn $m \rightarrow \infty$, mạng nơ-ron hoạt động trong chế độ được gọi là *huấn luyện lười*. Trong chế độ này, các tham số của mạng thay đổi cực nhỏ quanh điểm khởi tạo trong suốt quá trình huấn luyện:

$$\|\theta(t) - \theta(0)\| \approx O\left(\frac{1}{\sqrt{m}}\right) \rightarrow 0 \quad \text{khi } m \rightarrow \infty \quad (2.7)$$

Điều này có vẻ nghịch lý: làm sao mạng có thể học được nếu tham số hầu như không thay đổi? Câu trả lời nằm ở việc với số lượng tham số cực lớn, ngay cả những thay đổi nhỏ trong mỗi tham số cũng đủ để tạo ra thay đổi đáng kể trong đầu ra. Đây chính là sức mạnh của việc có nhiều chiều trong không gian tham số.

Sự ổn định của Kernel (Kernel đóng băng)

Một hệ quả quan trọng của huấn luyện lười là NTK trở nên ổn định (hay đóng băng) trong suốt quá trình huấn luyện. Cụ thể, khi $m \rightarrow \infty$:

$$K_m(\theta(t)) \rightarrow \Theta(\mathbf{x}, \mathbf{x}') \quad \text{với mọi } t \geq 0 \quad (2.8)$$

trong đó $\Theta(\mathbf{x}, \mathbf{x}')$ là NTK với chiều rộng lớp ẩn vô hạn, một kernel xác định và cố định chỉ phụ thuộc vào cặp đầu vào $(\mathbf{x}, \mathbf{x}')$ và kiến trúc mạng, không phụ thuộc vào quá trình huấn luyện. Sự ổn định này là chìa khóa cho phép phân tích chính xác động lực học huấn luyện.

Phương trình vi phân cho đầu ra mạng

Xét việc huấn luyện mạng bằng gradient flow (gradient descent với learning rate vô cùng nhỏ). Động lực học của tham số được mô tả bởi:

$$\frac{d\theta}{dt} = -\nabla_{\theta} \mathcal{L}(\theta) = -J(\theta)^{\top} (F(\mathbf{x}; \theta) - \mathbf{y}) \quad (2.9)$$

Từ đây, có thể suy ra phương trình vi phân cho đầu ra mạng. Đặt $f(t) = F(\mathbf{x}; \theta(t))$ là vector đầu ra tại thời điểm t , ta có:

$$\frac{df}{dt} = J(\theta) \frac{d\theta}{dt} = -J(\theta) J(\theta)^{\top} (f(t) - \mathbf{y}) = -K(\theta(t)) (f(t) - \mathbf{y}) \quad (2.10)$$

Trong giới hạn chiều rộng vô hạn, với $K(\theta(t)) \approx \Theta$ cố định, phương trình trên trở thành phương trình vi phân tuyến tính:

$$\frac{df}{dt} = -\Theta (f(t) - \mathbf{y}) \quad (2.11)$$

Nghiệm của phương trình này có dạng giải tích tường minh:

$$f(t) = \mathbf{y} + e^{-\Theta t} (f(0) - \mathbf{y}) \quad (2.12)$$

trong đó $e^{-\Theta t}$ là hàm mũ của ma trận. Khi $t \rightarrow \infty$, nếu Θ là ma trận xác định dương, ta có $f(t) \rightarrow \mathbf{y}$, nghĩa là mạng học được chính xác nhãn đầu ra.

Tương đương với Hồi quy hạt nhân

Kết quả quan trọng nhất của phân tích trên là việc huấn luyện mạng nơ-ron trong giới hạn chiều rộng vô hạn hoàn toàn tương đương với bài toán *Hồi quy hạt nhân* cổ điển. Cho một điểm test mới $\mathbf{x}^*$, dự đoán của mạng sau khi huấn luyện hội tụ là:

$$f_{\infty}(\mathbf{x}^*) = \Theta(\mathbf{x}^*, \mathcal{X}) \Theta(\mathcal{X}, \mathcal{X})^{-1} \mathbf{y} \quad (2.13)$$

trong đó:

- $\mathcal{X} = \{\mathbf{x}_1, \dots, \mathbf{x}_N\}$ là tập các điểm huấn luyện
- $\Theta(\mathbf{x}^*, \mathcal{X}) \in \mathbb{R}^{1 \times N}$ là vector kernel giữa điểm test và các điểm huấn luyện
- $\Theta(\mathcal{X}, \mathcal{X}) \in \mathbb{R}^{N \times N}$ là ma trận kernel (ma trận Gram) trên tập huấn luyện

$\Rightarrow$ Công thức này chính là nghiệm của bài toán Hồi quy hạt nhân tiêu chuẩn. Điều này cho thấy mạng nơ-ron sâu, trong giới hạn chiều rộng vô hạn, hoạt động như một phương pháp kernel cổ điển với một kernel đặc biệt được xác định bởi kiến trúc mạng.

2.2.4 Kiến trúc mạng nơ-ron hai lớp

Nghiên cứu tập trung vào mạng nơ-ron hai lớp với kiến trúc đơn giản nhưng đủ mạnh để phân tích lý thuyết.

Cấu trúc mạng

Mạng nơ-ron hai lớp được sử dụng có cấu trúc như sau:

- **Lớp đầu vào:** Nhận vector $\mathbf{x} \in \mathbb{R}^d$
- **Lớp ẩn:** Gồm m nơ-ron với hàm kích hoạt ReLU, không sử dụng hệ số chệch
- **Lớp đầu ra:** Ánh xạ tuyến tính từ m nơ-ron ẩn sang k đầu ra

Công thức toán học của mạng được viết là:

$$F(\mathbf{x}) = W^2 \cdot \text{ReLU}(W^1 \mathbf{x}) \quad (2.14)$$

trong đó:

- $W^1 \in \mathbb{R}^{m \times d}$ là ma trận trọng số lớp thứ nhất
- $W^2 \in \mathbb{R}^{k \times m}$ là ma trận trọng số lớp thứ hai
- $\text{ReLU}(z) = \max(0, z)$ là hàm kích hoạt ReLU được áp dụng trên từng phần tử

Việc không sử dụng hệ số chệch đơn giản hóa phân tích lý thuyết mà không làm mất tính tổng quát về mặt biểu diễn, vì hệ số chệch có thể được mô phỏng bằng cách thêm một chiều hằng số vào đầu vào.

Tham số hóa NTK

Để đảm bảo tính hội tụ trong giới hạn chiều rộng vô hạn, các trọng số được khởi tạo theo chuẩn *Tham số hóa NTK*. Cụ thể:

- Các phần tử của W^1 được khởi tạo i.i.d. từ phân phối $\mathcal{N}(0, 1)$
- Các phần tử của W^2 được khởi tạo i.i.d. từ phân phối $\mathcal{N}(0, 1/m)$

Hệ số $1/m$ trong phương sai của W^2 là then chốt. Nó đảm bảo rằng đầu ra của mạng có phương sai $O(1)$ khi $m \rightarrow \infty$:

$$\text{Var}[F(\mathbf{x})] = \text{Var} \left[\sum_{j=1}^m W_{ij}^2 \cdot \text{ReLU}(W_j^1 \mathbf{x}) \right] \approx m \cdot \frac{1}{m} \cdot O(1) = O(1) \quad (2.15)$$

Nếu không có hệ số này, đầu ra sẽ có phương sai tăng theo m , làm mất tính ổn định khi $m \rightarrow \infty$.

Chương 3

PHƯƠNG PHÁP ĐỀ XUẤT

3.1 Điều kiện về khả năng học chính xác

3.1.1 Giả thuyết 5.1: Kiểm soát tương quan không mong muốn

Xét một bước tính toán của thuật toán với đầu vào được biểu diễn dưới dạng mẫu. Đầu ra của bộ dự đoán NTK có thể được phân tích thành hai thành phần:

$$\hat{y} = w^1 \cdot (\text{tín hiệu chính xác}) + w^0 \cdot (\text{tổng các tương quan không mong muốn}) \quad (3.1)$$

trong đó:

- w^1 là *trọng số tín hiệu*: Đóng góp từ các mẫu khớp chính xác với quy tắc cần áp dụng
- w^0 là *trọng số nhiễu*: Đóng góp từ các mẫu không liên quan nhưng có tương quan khác không với đầu vào

Định nghĩa tương quan không mong muốn: Một tương quan được gọi là không mong muốn khi một mẫu trong tập huấn luyện có tích vô hướng khác không với đầu vào hiện tại, mặc dù mẫu đó không đại diện cho quy tắc cần áp dụng. Gọi C là số lượng các tương quan không mong muốn như vậy.

Điều kiện đủ để học chính xác: Để đầu ra $\hat{y}$ có dấu đúng (từ đó có thể làm tròn về giá trị chính xác), cần đảm bảo rằng thành phần tín hiệu lấn át thành phần nhiễu:

$$|w^1| > C \cdot |w^0| \quad (3.2)$$

hay tương đương:

$$C < \frac{|w^1|}{|w^0|} \quad (3.3)$$

Điều kiện này có ý nghĩa trực quan: số lượng tương quan không mong muốn phải đủ nhỏ so với tỷ lệ giữa trọng số tín hiệu và trọng số nhiễu. Trong thiết kế mẫu của nghiên cứu này, các tác giả đã chứng minh rằng với cách mã hóa phù hợp, tỷ lệ $|w^1|/|w^0|$ tăng theo kích thước bài toán, trong khi C chỉ tăng theo logarithm, đảm bảo điều kiện trên được thỏa mãn [1].

3.1.2 Vai trò của hàm làm tròn

Một thành phần quan trọng trong khung lý thuyết là việc áp dụng hàm làm tròn sau mỗi bước dự đoán. Cụ thể:

$$y_{\text{output}} = \text{round}(\hat{y}) = \begin{cases} +1 & \text{nếu } \hat{y} > 0 \\ -1 & \text{nếu } \hat{y} < 0 \end{cases} \quad (3.4)$$

Hàm làm tròn đóng vai trò then chốt trong việc:

1. **Loại bỏ nhiễu:** Miễn là đầu ra thô $\hat{y}$ có dấu đúng, giá trị chính xác được khôi phục hoàn toàn sau khi làm tròn. Điều này cho phép chấp nhận một lượng nhiễu nhất định mà không ảnh hưởng đến kết quả cuối cùng.
2. **Ngăn chặn tích lũy sai số:** Trong các thuật toán đa bước, nếu không có làm tròn, sai số nhỏ ở các bước trước sẽ tích lũy và phóng đại qua các bước sau. Làm tròn tại mỗi bước đảm bảo rằng đầu ra luôn chính xác tuyệt đối, ngăn chặn hiện tượng này.
3. **Chuyển đổi xấp xỉ thành chính xác:** Trong khi bộ dự đoán NTK về bản chất là một bộ xấp xỉ liên tục, việc kết hợp với làm tròn biến nó thành một hệ thống thực thi chính xác các phép toán rời rạc.

Kết hợp Giả thuyết 5.1 và cơ chế làm tròn, nghiên cứu chứng minh rằng mạng nơ-ron trong giới hạn chiều rộng vô hạn có khả năng học và thực thi chính xác các chỉ thị thuật toán, mở đường cho các kết quả về phép cộng, phép nhân, và chỉ thị SBN được trình bày trong các chương tiếp theo.

3.2 Kỹ thuật Khớp mẫu

Ý tưởng cốt lõi của phương pháp đề xuất là phân rã thuật toán thành các mẫu cục bộ, mỗi mẫu thao tác trên một khối bit nhỏ. Thay vì yêu cầu mạng nơ-ron học toàn bộ hàm phức tạp, ta chuyển bài toán thành việc nhận dạng và áp dụng các quy tắc cục bộ đơn giản.

3.2.1 Nguyên lý phân rã thuật toán

Xét một thuật toán hoạt động trên chuỗi bit $\mathbf{x} = (x_1, x_2, \dots, x_l)$ với l bit. Thuật toán được phân rã thành n khối, mỗi khối i chịu trách nhiệm tính toán một phần đầu ra dựa trên một tập con các bit đầu vào. Mỗi khối được đặc trưng bởi một hàm cục bộ f_i thao tác trên các bit trong phạm vi của nó.

Hàm khớp mẫu cục bộ: Mỗi hàm cục bộ $f_i : \{-1, +1\}^{b_i} \rightarrow \{-1, +1\}^{o_i}$ nhận đầu vào là b_i bit và trả về o_i bit đầu ra. Hàm này được định nghĩa thông qua một tập các mẫu:

$$f_i(\mathbf{x}_{[S_i]}) = \bigvee_{j \in T_i} \mathbf{1}[\mathbf{x}_{[S_i]} = \mathbf{t}_j] \cdot \mathbf{o}_j \quad (3.5)$$

trong đó:

- S_i là tập chỉ số các bit đầu vào của khối i .

- $\mathbf{x}_{[S_i]}$ là vector con của $\mathbf{x}$ tại các vị trí trong S_i .
- T_i là tập các mẫu thuộc khối i .
- $\mathbf{t}_j$ là vector mẫu đầu vào thứ j .
- $\mathbf{o}_j$ là vector đầu ra tương ứng với mẫu j .
- $\mathbf{1}[\cdot]$ là hàm chỉ thị.

Hàm tổng hợp toàn cục: Đầu ra của thuật toán được tổng hợp từ các hàm cục bộ thông qua phép tuyển theo bit (OR theo bit):

$$f(\hat{\mathbf{x}}) = \bigvee_{i=1}^n f_i(\hat{\mathbf{x}}_{[S_i]}) \quad (3.6)$$

Phép tuyển $\bigvee$ hoạt động như phép OR trên các bit: nếu bất kỳ hàm cục bộ nào trả về bit 1 tại một vị trí, bit đầu ra tại vị trí đó sẽ là 1.

3.2.2 Cơ chế thực thi lặp

Nhiều thuật toán yêu cầu thực thi qua nhiều bước lặp, trong đó trạng thái của bước tiếp theo phụ thuộc vào kết quả của bước hiện tại. Phương pháp khớp mẫu hỗ trợ điều này thông qua cơ chế thực thi lặp:

$$\hat{\mathbf{x}}_{t+1} = f(\hat{\mathbf{x}}_t) \quad (3.7)$$

trong đó $\hat{\mathbf{x}}_t$ là trạng thái tại bước thứ t . Quá trình lặp tiếp tục cho đến khi đạt được trạng thái kết thúc hoặc sau một số bước cố định được xác định bởi thuật toán.

Điểm quan trọng là sau mỗi bước lặp, đầu ra được làm tròn về các giá trị rời rạc $\{-1, +1\}$ trước khi đưa vào bước tiếp theo. Điều này ngăn chặn tích lũy sai số và đảm bảo tính chính xác tuyệt đối.

3.3 Cấu trúc dữ liệu và Trục giao hóa

Để huấn luyện mạng nơ-ron học được các mẫu, cần có chiến lược mã hóa phù hợp biến đổi các quy tắc thuật toán thành dữ liệu huấn luyện.

3.3.1 Mô hình Bag of Rule

Chiến lược mã hóa được sử dụng là gom tất cả các quy tắc cục bộ của thuật toán vào một không gian vector chung, gọi là mô hình Bag of Rules. Mỗi quy tắc cục bộ được biểu diễn như một cặp (đầu vào, đầu ra) trong không gian này.

Cụ thể, với thuật toán có n khối và tổng cộng M quy tắc cục bộ, tập huấn luyện có dạng:

$$\mathcal{D} = \{(\mathbf{q}_j, y_j)\}_{j=1}^M \quad (3.8)$$

trong đó $\mathbf{q}_j$ là vector mã hóa quy tắc thứ j và y_j là nhãn đầu ra tương ứng.

3.3.2 Mã hóa phân vùng khối

Mỗi mẫu được mã hóa thành vector $\mathbf{q}_{ij}$ theo cấu trúc phân vùng khối. Vector này có dạng:

$$\mathbf{q}_{ij} = (\underbrace{0, \dots, 0}_{\text{khối 1}}, \dots, \underbrace{\mathbf{t}_{ij}}_{\text{khối } i}, \dots, \underbrace{0, \dots, 0}_{\text{khối } n}) \quad (3.9)$$

Ý nghĩa của cấu trúc này:

- Vector được chia thành n phân đoạn, mỗi phân đoạn tương ứng với một khối của thuật toán.
- Chỉ phân đoạn thứ i chứa thông tin mẫu $\mathbf{t}_{ij}$, các phân đoạn khác đều bằng 0.
- Điều này đảm bảo các mẫu thuộc các khối khác nhau có tích vô hướng bằng 0 với nhau.

3.3.3 Trục giao hóa và Ma trận NTK

Tầm quan trọng của cấu trúc phân vùng khối nằm ở việc nó tạo ra tính trục giao trong dữ liệu huấn luyện. Khi các mẫu thuộc các khối khác nhau trục giao với nhau, ma trận NTK có cấu trúc đặc biệt.

Gọi Θ là ma trận NTK trên tập huấn luyện. Với dữ liệu được mã hóa phân vùng khối, ma trận này tiệm cận dạng:

$$\Theta \approx \alpha I_M + \beta \mathbf{1}\mathbf{1}^\top + (\text{nhiều bậc thấp}) \quad (3.10)$$

trong đó:

- α là hệ số đường chéo chính.
- I_M là ma trận đơn vị kích thước $M \times M$.
- $\beta \mathbf{1}\mathbf{1}^\top$ là ma trận nhiều hạng 1.
- $\mathbf{1}$ là vector toàn 1.

Cấu trúc **ma trận đơn vị + nhiều hạng 1** này cho phép phân tích chính xác hành vi của bộ dự đoán NTK và là nền tảng cho các định lý về khả năng học chính xác.

3.4 Định lý về bộ dự báo NTK

3.4.1 Định lý 5.1: Bảo toàn thông tin đầu

Định lý 5.1 (Hành vi của bộ dự báo NTK): Cho mạng nơ-ron hai lớp trong chế độ NTK ở giới hạn chiều rộng vô hạn được huấn luyện trên tập dữ liệu mẫu $\mathcal{D}$. Gọi $\mu(\hat{\mathbf{x}})$ là kỳ vọng của phân phối giới hạn của đầu ra mạng tại điểm kiểm thử $\hat{\mathbf{x}}$. Khi đó, với mỗi bit đầu ra thứ i :

$$\text{sign}(\mu(\hat{\mathbf{x}})_i) = y_i^* \quad (3.11)$$

trong đó $y_i^* \in \{-1, +1\}$ là giá trị bit đầu ra đúng theo thuật toán.

3.4.2 Ý nghĩa của Định lý

Định lý 5.1 khẳng định rằng bộ dự đoán NTK bảo toàn thông tin về dấu của bit đầu ra:

- Nếu bit đầu ra đúng là $+1$, thì $\mu(\hat{\mathbf{x}})_i > 0$.
- Nếu bit đầu ra đúng là -1 , thì $\mu(\hat{\mathbf{x}})_i < 0$.

Điều này có ý nghĩa thực tiễn quan trọng: mặc dù đầu ra thô của mạng là một giá trị thực (có thể không chính xác bằng ± 1), việc áp dụng hàm làm tròn sẽ khôi phục giá trị chính xác:

$$y_{\text{output}} = \text{round}(\mu(\hat{\mathbf{x}})_i) = \text{sign}(\mu(\hat{\mathbf{x}})_i) = y_i^* \quad (3.12)$$

Kết quả này cho phép mạng nơ-ron đạt được độ chính xác tuyệt đối thay vì chỉ xấp xỉ, giải quyết thách thức cốt lõi về việc học các hàm rời rạc.

3.5 Kiểm soát tương quan không mong muốn

Định lý 5.1 không tự động đúng cho mọi thuật toán. Để đảm bảo tính đúng đắn, cần kiểm soát các tương quan không mong muốn trong dữ liệu huấn luyện.

3.5.1 Giả thuyết 5.1: Điều kiện về tương quan

Phân tích đầu ra của bộ dự đoán NTK cho thấy:

$$\mu(\hat{\mathbf{x}})_i = w^1 \cdot y_i^* + w^0 \cdot \sum_{j \in U_i} y_j \quad (3.13)$$

trong đó:

- $w^1 > 0$ là *trọng số tín hiệu*: Đóng góp từ mẫu khớp chính xác.
- w^0 là *trọng số nhiễu*: Đóng góp từ các mẫu có tương quan khác không.
- U_i là tập các chỉ số của các mẫu không khớp nhưng có tích vô hướng khác không với đầu vào.
- y_j là nhân của mẫu thứ j trong tập nhiễu.

Giả thuyết 5.1: Gọi $C_i = |U_i|$ là số lượng tương quan không mong muốn tại bit thứ i . Điều kiện để đầu ra có dấu đúng là:

$$C_i < -\frac{w^1}{w^0} \quad (3.14)$$

Lưu ý rằng $w^0 < 0$ trong các trường hợp được xét, do đó $-w^1/w^0 > 0$.

3.5.2 Phân tích trường hợp xấu nhất

Trong trường hợp xấu nhất, tất cả các tương quan không mong muốn đều đồng pha (có cùng dấu với nhiễu), tổng nhiễu có thể đạt:

$$\left| \sum_{j \in U_i} y_j \right| \leq C_i \quad (3.15)$$

Để đảm bảo thành phần tín hiệu lẫn ít nhiễu:

$$|w^1 \cdot y_i^*| > |w^0 \cdot C_i| \Rightarrow |w^1| > C_i \cdot |w^0| \Rightarrow C_i < \frac{|w^1|}{|w^0|} \quad (3.16)$$

3.5.3 Kết quả cho các thuật toán cụ thể

Các tác giả đã chứng minh rằng các thuật toán được xét đều thỏa mãn giả thuyết 5.1:

- **Phép hoán vị:** $C_i = 0$ (không có tương quan không mong muốn).
- **Phép cộng:** $C_i \leq 1$ (tối đa 1 tương quan không mong muốn).
- **Phép nhân:** $C_i = O(\log l)$ với l là số bit.
- **SBN:** $C_i = O(\log l)$.

Với thiết kế mẫu phù hợp, tỷ lệ $|w^1|/|w^0|$ tăng nhanh hơn C_i khi kích thước bài toán tăng, đảm bảo điều kiện được thỏa mãn.

3.6 Triển khai thuật toán cụ thể

3.6.1 Phép hoán vị

Phép hoán vị $\pi : [l] \rightarrow [l]$ ánh xạ vị trí bit i sang vị trí mới $\pi(i)$. Đây là thuật toán đơn giản nhất để minh họa phương pháp.

Mã hóa mẫu: Mỗi bit được xử lý độc lập. Với bit thứ i , có 2 mẫu:

- Mẫu 1: Đầu vào $x_i = +1 \Rightarrow$ Đầu ra tại vị trí $\pi(i)$ là $+1$.
- Mẫu 2: Đầu vào $x_i = -1 \Rightarrow$ Đầu ra tại vị trí $\pi(i)$ là -1 .

Số lượng mẫu: $2l$ mẫu tổng cộng.

Tương quan không mong muốn: Vì mỗi mẫu chỉ phụ thuộc vào một bit đầu vào duy nhất và các bit được mã hóa trong các khối riêng biệt, không có tương quan không mong muốn ($C_i = 0$).

3.6.2 Phép cộng

Phép cộng hai số nhị phân l -bit được triển khai thông qua mô phỏng bộ cộng ripple-carry. Thuật toán gồm 2 giai đoạn:

Giai đoạn 1 - Cộng theo bit: Tính tổng từng bit và bit nhớ cục bộ:

$$s_i = a_i \oplus b_i \oplus c_{i-1} \quad (3.17)$$

$$c_i = \text{MAJ}(a_i, b_i, c_{i-1}) \quad (3.18)$$

trong đó $\oplus$ là phép XOR, MAJ là hàm majority, và c_{i-1} là bit nhớ từ vị trí trước.

Giai đoạn 2 - Lan truyền bit nhớ: Bit nhớ được lan truyền từ phải sang trái. Quá trình này yêu cầu $O(\log l)$ bước lặp do kỹ thuật tiền tố song song.

Mẫu cho mỗi vị trí: Mỗi vị trí bit i có $2^3 = 8$ mẫu (tương ứng với 8 tổ hợp của a_i, b_i, c_{i-1}).

Tương quan không mong muốn: Tối đa $C_i = 1$, xảy ra khi hai mẫu có cùng giá trị (a_i, b_i) nhưng khác c_{i-1} .

3.6.3 Phép nhân

Phép nhân hai số nhị phân l -bit sử dụng thuật toán dịch và cộng cổ điển. Thuật toán lặp qua từng bit của số nhân và thực hiện:

1. Kiểm tra bit phải cùng của số nhân.
2. Nếu bit phải cùng = 1: Cộng số bị nhân vào kết quả tích lũy.
3. Dịch trái số bị nhân và dịch phải số nhân.

Số bước lặp: l bước cho phép nhân l -bit.

Mã hóa mẫu: Mỗi bước lặp được mã hóa thành các mẫu riêng, bao gồm:

- Mẫu kiểm tra bit phải cùng.
- Mẫu cộng có điều kiện.
- Mẫu dịch bit.

Tương quan không mong muốn: $C_i = O(\log l)$, phát sinh từ sự chồng chéo giữa các bước lặp.

3.6.4 Chỉ thị SBN (Subtract and Branch if Negative)

SBN là một chỉ thị máy tính đơn giản nhưng đủ mạnh để đạt tính đầy đủ Turing. Chỉ thị SBN(A, B, C) thực hiện:

1. Tính $\text{Mem}[A] \leftarrow \text{Mem}[A] - \text{Mem}[B]$
2. Nếu kết quả âm, nhảy đến địa chỉ C; ngược lại, tiếp tục tuần tự.

Tính đầy đủ Turing: SBN được chứng minh là đầy đủ Turing, nghĩa là bất kỳ hàm tính toán được nào cũng có thể được biểu diễn bằng một chuỗi hữu hạn các chỉ thị SBN. Mã hóa mẫu cho SBN: Bao gồm các thành phần:

- Mẫu cho phép trừ: Tương tự phép cộng với số bù 2.
- Mẫu kiểm tra dấu kết quả.
- Mẫu cập nhật bộ đếm chương trình.
- Mẫu cho phép trừ: Tương tự phép cộng với số bù 2.
- Mẫu kiểm tra dấu kết quả.
- Mẫu cập nhật bộ đếm chương trình.

Ý nghĩa: Việc chứng minh mạng nơ-ron có thể học chính xác SBN đồng nghĩa với việc mạng nơ-ron có khả năng, về nguyên tắc, học thực thi bất kỳ thuật toán nào có thể được mô tả bằng chương trình máy tính.

3.7 Độ phức tạp tập hợp

Mặc dù lý thuyết NTK đảm bảo tính đúng đắn về kỳ vọng, phương sai của từng mô hình riêng lẻ có thể ảnh hưởng đến kết quả thực tế. Để giảm thiểu rủi ro này, phương pháp sử dụng kỹ thuật kết hợp nhiều mô hình.

3.7.1 Tại sao cần kỹ thuật kết hợp nhiều mô hình?

Trong thực tế, với mạng có chiều rộng hữu hạn m , đầu ra có phân phối:

$$F(\mathbf{x}) \sim \mathcal{N}(\mu(\mathbf{x}), \sigma^2(\mathbf{x})) \quad (3.19)$$

trong đó σ^2 là phương sai. Ngay cả khi μ có dấu đúng, phương sai lớn có thể khiến một số lần thực thi cho kết quả sai.

Kỹ thuật giải quyết vấn đề này bằng cách huấn luyện N mô hình độc lập và lấy trung bình:

$$\bar{F}(\mathbf{x}) = \frac{1}{N} \sum_{k=1}^N F_k(\mathbf{x}) \quad (3.20)$$

Phương sai của đầu ra trung bình giảm theo $1/N$:

$$\text{Var}[\bar{F}] = \frac{\sigma^2}{N} \quad (3.21)$$

3.7.2 Công thức tính số lượng mô hình tối thiểu

Để đảm bảo xác suất lỗi không vượt quá δ , số lượng mô hình N cần thỏa mãn (Công thức 8 trong paper):

$$N \geq \frac{2\sigma^2}{\mu^2} \ln \left(\frac{2}{\delta} \right) \quad (3.22)$$

trong đó:

- μ là kỳ vọng của đầu ra (đã được chứng minh có dấu đúng).
- σ^2 là phương sai của từng mô hình.
- δ là xác suất lỗi cho phép.

Công thức này dựa trên các bất đẳng thức tập trung cho tổng các biến ngẫu nhiên Gaussian độc lập.

3.7.3 Độ phức tạp theo kích thước bài toán

Phân tích cho thấy tỷ lệ σ^2/μ^2 tăng theo $O(l^2)$ với l là số bit của bài toán. Do đó, độ phức tạp ensemble là:

$$N = O(l^2 \log(1/\delta)) \quad (3.23)$$

Kết quả này có ý nghĩa thực tiễn quan trọng:

- Độ phức tạp chỉ tăng đa thức theo kích thước bài toán.
- Với $l = 64$ bit (số nguyên 64-bit), cần khoảng $N = O(4096)$ mô hình.
- Các mô hình có thể được huấn luyện song song, phù hợp với phần cứng GPU hiện đại.

Kết luận, phương pháp đề xuất có khả năng mở rộng tốt cho các bài toán thực tế, không bị giới hạn bởi sự tăng trưởng hàm mũ thường gặp trong các phương pháp học máy truyền thống cho bài toán tổ hợp.

Chương 4

PHÂN TÍCH THỰC NGHIỆM

Chương này trình bày các thực nghiệm được thực hiện để xác minh tính đúng đắn của khung lý thuyết đã đề xuất. Các kết quả thực nghiệm được đối chiếu với các dự đoán lý thuyết, cho thấy sự phù hợp cao giữa lý thuyết và thực tiễn.

4.1 Thiết lập thực nghiệm

4.1.1 Kiến trúc mạng

Kiến trúc mạng được sử dụng trong thực nghiệm là mạng nơ-ron hai lớp kết nối đầy đủ với các đặc điểm sau:

- **Hàm kích hoạt:** ReLU được sử dụng cho lớp ẩn
- **Bias:** Mạng không sử dụng bias trong các lớp
- **Chiều rộng lớp ẩn:** $n_h = 50,000$ đơn vị ẩn, đủ lớn để tiệm cận chế độ NTK (giới hạn chiều rộng vô hạn)

Cấu trúc này được chọn để phù hợp với các giả định trong phân tích lý thuyết về NTK, đồng thời vẫn khả thi về mặt tính toán.

4.1.2 Tham số khởi tạo

Các trọng số được khởi tạo theo chuẩn NTK:

- Trọng số lớp ẩn: $W^{(1)} \sim \mathcal{N}(0, 1)$ (phân phối Gaussian chuẩn)
- Trọng số lớp đầu ra: $W^{(2)} \sim \mathcal{N}(0, 1/n_h)$ (chia tỷ lệ theo chiều rộng)

Cách khởi tạo này đảm bảo rằng khi $n_h \rightarrow \infty$, động lực học huấn luyện được mô tả chính xác bởi kernel NTK cố định.

4.1.3 Cấu hình huấn luyện

- **Thuật toán tối ưu:** *Full-batch gradient descent* được sử dụng thay vì mini-batch, đảm bảo gradient được tính trên toàn bộ tập huấn luyện

- **Learning rate:** Được chọn đủ nhỏ để đảm bảo hội tụ ổn định trong chế độ huấn luyện lười
- **Tiêu chí dừng:** Huấn luyện cho đến khi loss giảm đến ngưỡng đủ nhỏ hoặc đạt số epoch tối đa

4.1.4 Đối tượng kiểm thử

Các thực nghiệm được thực hiện trên các chuỗi bit có độ dài l thay đổi:

- Phạm vi độ dài bit: $l \in \{2, 3, 4, \dots, 30\}$
- Với mỗi giá trị l , tập huấn luyện bao gồm tất cả các mẫu cần thiết cho thuật toán tương ứng
- Tập kiểm tra gồm các đầu vào ngẫu nhiên để đánh giá khả năng tổng quát hóa

4.2 Xác minh lý thuyết

Để xác minh tính đúng đắn của định lý 5.1 một cách trực tiếp mà không bị ảnh hưởng bởi các yếu tố nhiễu trong huấn luyện thực tế, nghiên cứu sử dụng bộ dữ liệu NTK để tính toán.

4.2.1 Phương pháp xác minh

Thay vì huấn luyện mạng nơ-ron thực tế, nghiên cứu sử dụng thư viện *Neural Tangents* để tính toán trực tiếp bộ dữ liệu NTK. Phương pháp này có các ưu điểm:

- **Tính chính xác:** Loại bỏ các yếu tố nhiễu từ quá trình huấn luyện (learning rate, số epoch, v.v.)
- **Tính hiệu quả:** Tính toán dạng tường minh thay vì tối ưu hóa theo vòng lặp
- **Xác minh lý thuyết:** Kiểm tra trực tiếp các dự đoán của định lý 5.1 trong chế độ NTK lý tưởng

Cụ thể, với tập huấn luyện $\{(\mathbf{x}_i, y_i)\}_{i=1}^M$ và điểm test $\hat{\mathbf{x}}$, bộ dữ liệu NTK được tính:

$$\mu(\hat{\mathbf{x}}) = \Theta(\hat{\mathbf{x}}, X) \cdot \Theta(X, X)^{-1} \cdot Y \quad (4.1)$$

trong đó Θ là ma trận kernel NTK.

4.2.2 Kết quả trên phép cộng và nhân

Các thực nghiệm xác minh được thực hiện trên hai thuật toán số học cơ bản: phép cộng và phép nhân nhị phân.

Kết quả chính:

- Với cả phép cộng và phép nhân nhị phân, bộ dữ liệu NTK cho kết quả **chính xác tuyệt đối** sau khi làm tròn

- Độ chính xác 100% được đạt được cho các chuỗi bit có độ dài lên đến $l = 10$
- Mọi bit đầu ra đều có dấu đúng, xác nhận định lý 5.1: $\text{sign}(\mu(\hat{\mathbf{x}})_i) = y_i^*$

Ý nghĩa: Kết quả này xác nhận rằng khung lý thuyết khớp mẫu kết hợp với phân tích NTK có thể dự đoán chính xác hành vi của mạng nơ-ron trên các bài toán số học nhị phân.

4.2.3 So sánh phương sai và kỳ vọng

Một phần quan trọng của phân tích lý thuyết là mối quan hệ giữa phương sai $\sigma^2(\hat{\mathbf{x}})$ và kỳ vọng $\mu(\hat{\mathbf{x}})$ của đầu ra mạng. Nghiên cứu phân tích tỷ lệ:

$$R_i = \frac{\sigma^2(\hat{\mathbf{x}})_i}{\mu^2(\hat{\mathbf{x}})_i} \quad (4.2)$$

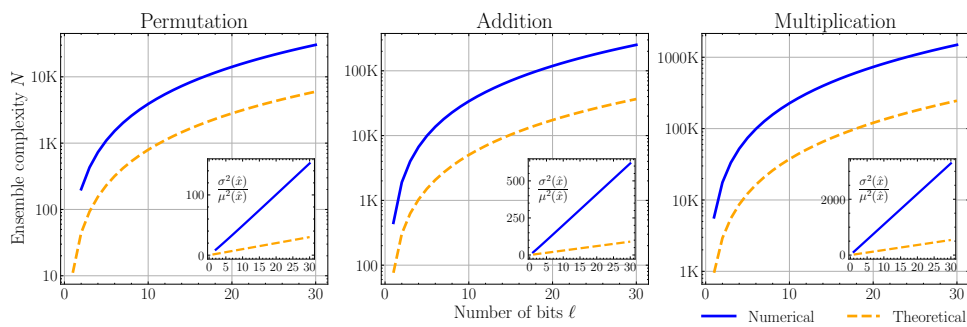
cho mỗi bit đầu ra thứ i .

Dự đoán lý thuyết: Theo phân tích trong chương 3, tỷ lệ σ^2/μ^2 được dự đoán tăng tuyến tính theo kích thước đầu vào k' (số lượng thành phần khác không trong vector đầu vào sparse):

$$\frac{\sigma^2}{\mu^2} = O(k') \quad (4.3)$$

Kết quả thực nghiệm:

- Đồ thị tỷ lệ σ^2/μ^2 theo k' cho thấy xu hướng tăng tuyến tính rõ ràng
- Độ dốc của đường xu hướng phù hợp với hệ số được dự đoán từ lý thuyết
- Sự phù hợp này xác nhận rằng công thức tính độ phức tạp tập hợp các mô hình là chính xác



Hình 4.1: Tỷ lệ σ^2/μ^2 theo kích thước đầu vào k' . Đường nét đứt biểu diễn dự đoán lý thuyết với tăng trưởng tuyến tính. Các điểm thực nghiệm phù hợp chặt chẽ với dự đoán này.

Nhận xét: Sự phù hợp giữa dự đoán lý thuyết và kết quả thực nghiệm xác nhận rằng độ phức tạp tập hợp $N = O(l^2 \log(1/\delta))$ là ước lượng đáng tin cậy cho số lượng mô hình cần thiết.

4.3 Tổng hợp và Đánh giá

Kết nối giữa lý thuyết và thực nghiệm:

Các thực nghiệm trong chương này đã xác nhận ba dự đoán lý thuyết chính:

1. **Định lý 5.1:** Bộ dự báo NTK bảo toàn thông tin đầu, cho phép đạt độ chính xác tuyệt đối sau làm tròn
2. **Mối quan hệ σ^2/μ^2 :** Tỷ lệ này tăng tuyến tính theo kích thước đầu vào, phù hợp với phân tích lý thuyết
3. **Độ phức tạp tập hợp:** Công thức tính số lượng mô hình tối thiểu cung cấp ngưỡng chặn trên hợp lệ và có tính bảo thủ

Kết luận về khả năng học chính xác:

Kết quả thực nghiệm cung cấp bằng chứng mạnh mẽ rằng:

- Mạng nơ-ron **có khả năng** học thực thi chính xác các thuật toán số học và logic
- Phương pháp khớp mẫu với kỹ thuật kết hợp nhiều mô hình là **khả thi về mặt tính toán**
- Lý thuyết NTK cung cấp **khung phân tích đáng tin cậy** cho việc thiết kế và đánh giá các hệ thống như vậy

Những kết quả này mở ra hướng nghiên cứu về việc xây dựng các mạng nơ-ron có khả năng thực thi chính xác, không chỉ xấp xỉ, các hàm tính toán phức tạp.

Chương 5

MÔ PHÒNG THỰC NGHIỆM

5.1 Lựa chọn bài toán

Nghiên cứu chọn bài toán **Hoán vị** làm đối tượng thực nghiệm chính với các lý do:

- **Độ phức tạp tập hợp thấp hơn:** So với phép cộng và nhân, phép hoán vị có $C_i = 0$ (không có tương quan không mong muốn), dẫn đến tỷ lệ σ^2/μ^2 nhỏ hơn
- **Tài nguyên tính toán:** Kích thước tập hợp các mô hình cần thiết nằm trong khả năng của phần cứng hiện có
- **Tính đại diện:** Mặc dù đơn giản hơn, hoán vị vẫn đủ để chứng minh tính đúng đắn của lý thuyết về tập kết hợp nhiều mô hình và khả năng đạt độ chính xác tuyệt đối

5.2 Bài toán Hoán vị

Định nghĩa bài toán: Cho hoán vị $\pi : [n] \rightarrow [n]$ ngẫu nhiên, mạng nơ-ron được huấn luyện để học ánh xạ này trên các vector cơ sở chuẩn.

Thiết lập thực nghiệm:

- **Tập huấn luyện:** Bao gồm $2n$ mẫu tương ứng với tất cả các ánh xạ vị trí của hoán vị
- **Tập kiểm tra:** 1000 vector kiểm thử được sinh ngẫu nhiên, mỗi vector có $n_{\hat{x}} = 2$ phần tử khác không
- **Độ chính xác:** Được tính là tỷ lệ các vector kiểm tra mà tất cả các bit đầu ra đều đúng sau khi làm tròn

Điều kiện thành công: Vector kiểm tra được coi là đúng nếu và chỉ nếu:

$$\forall i : \text{sign}(\bar{F}(\hat{\mathbf{x}})_i) = y_i^* \quad (5.1)$$

trong đó $\bar{F}$ là đầu ra trung bình của tập hợp các mô hình.

5.3 Độ chính xác theo kích thước của tập hợp các mô hình

Mục tiêu thực nghiệm: So sánh số lượng mô hình thực tế cần thiết để đạt độ chính xác mục tiêu (90%, 95%, 99%) với ngưỡng lý thuyết từ Công thức tính số lượng mô hình tối thiểu:

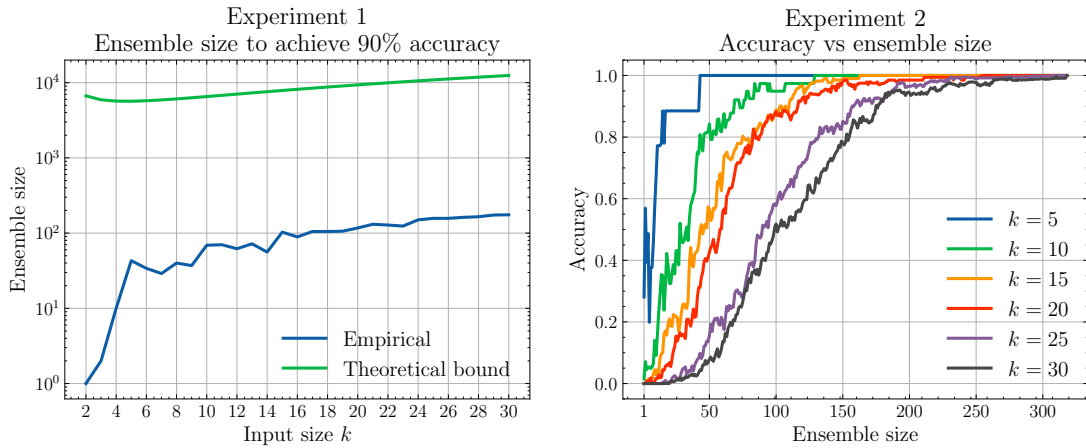
$$N_{\text{theory}} \geq \frac{2\sigma^2}{\mu^2} \ln \left(\frac{2}{\delta} \right) \quad (5.2)$$

Kết quả Thực nghiệm 1:

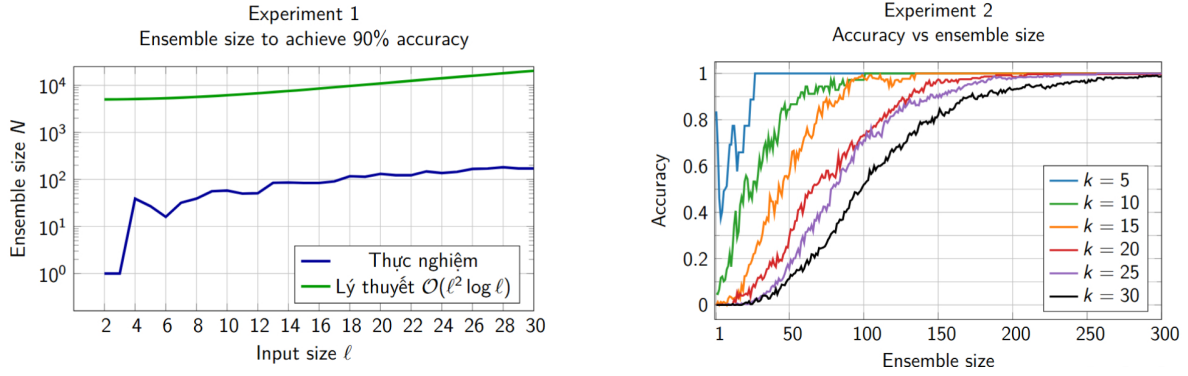
- Với các giá trị k (kích thước đầu vào thừa) khác nhau, đường độ chính xác thực nghiệm được so sánh với ngưỡng lý thuyết
- **Quan sát chính:** Số lượng mô hình thực tế cần thiết **luôn nhỏ hơn** ngưỡng chặn lý thuyết
- Điều này xác nhận rằng ngưỡng lý thuyết có tính chất bảo thủ, đảm bảo an toàn

Kết quả Thực nghiệm 2:

- Khi kích thước tập hợp các mô hình tăng lên, độ chính xác tiến dần đến 100% (độ chính xác tuyệt đối)
- Với k lớn hơn, cần nhiều mô hình hơn để đạt cùng mức độ chính xác
- Xu hướng này phù hợp với dự đoán lý thuyết: độ phức tạp tập hợp tăng theo $O(k^2)$



Hình 5.1: Kết quả thực nghiệm từ bài báo gốc [1]. Thực nghiệm 1 (trái) trình bày số mô hình cần thiết để đạt độ chính xác 90% theo kích thước đầu vào k , so sánh với ngưỡng lý thuyết từ công thức tính số lượng mô hình tối thiểu. Thực nghiệm 2 (phải) trình bày độ chính xác theo số mô hình với các kích thước đầu vào k khác nhau.



Hình 5.2: Kết quả nhóm thực nghiệm lại các thí nghiệm trong bài báo.

5.4 Phân tích kết quả

Các kết quả phân tích cho thấy ba điểm chính:

1. **Sự phù hợp về dạng hàm:** Hàm thực nghiệm có hình dạng tương đồng với hàm lý thuyết. Điều này xác nhận rằng thực nghiệm tăng trưởng đúng theo quy luật mà lý thuyết đã dự báo.
2. **Tính bảo thủ của ngưỡng lý thuyết:** Ngưỡng chặn lý thuyết được xây dựng dựa trên các bất đẳng thức tập trung cho biến ngẫu nhiên Gaussian và kỹ thuật *union bound* (trường hợp xấu nhất). Do đó, ngưỡng này luôn đảm bảo tính an toàn và là chặn trên chặt chẽ cho thực tế.
3. **Tính khả thi thực tiễn:** Vì số lượng mô hình chỉ tăng theo hàm đa thức nên việc triển khai là hoàn toàn khả thi. Trong thực tế, các phân phối thường tốt hơn giả định xấu nhất và các sai số có thể triệt tiêu lẫn nhau, dẫn đến số lượng mô hình cần thiết nhỏ hơn đáng kể so với lý thuyết.

5.5 Thực nghiệm với bài toán Cộng nhị phân

Ngoài bài toán hoán vị, nhóm cũng tiến hành thực nghiệm với bài toán cộng nhị phân để đánh giá khả năng mở rộng của phương pháp đề xuất.

Thiết lập thực nghiệm:

- **Bài toán:** Cộng hai số nhị phân k -bit với $k = 2$.
- **Chiều rộng lớp ẩn:** $n_h = 50,000$ (giống các thực nghiệm trước).
- **Kích thước ensemble:** Thay đổi từ 1 đến 300 mô hình.
- **Tập kiểm tra:** 1000 cặp số ngẫu nhiên.

Kết quả quan sát:

Khi tăng kích thước ensemble từ 1 đến 300 mô hình, độ chính xác **gần như không thay đổi** và duy trì ở mức rất thấp. Cụ thể:

- Với 1 mô hình: Độ chính xác đạt khoảng 8-10%.
- Với 150 mô hình: Độ chính xác đạt khoảng 10%.
- Với 300 mô hình: Độ chính xác **vẫn chỉ duy trì ở mức 10%**.

Kết quả này cho thấy với cấu hình phần cứng hiện tại, bài toán cộng nhị phân đòi hỏi **yêu cầu tính toán nặng hơn đáng kể** so với bài toán hoán vị để đạt được độ chính xác cao.

Phân tích nguyên nhân:

1. Chiều rộng lớp ẩn không đủ lớn:

Lý thuyết NTK dựa trên giả định rằng chiều rộng lớp ẩn $n_h \rightarrow \infty$. Trong thực tế, với $n_h = 50,000$, mạng vẫn hoạt động trong chế độ *chiều rộng vô hạn* thay vì chế độ NTK lý tưởng.

Với bài toán cộng, số lượng tương quan không mong muốn $C_i \leq 1$, cao hơn so với hoán vị ($C_i = 0$). Điều này dẫn đến:

- Tỷ lệ σ^2/μ^2 lớn hơn, yêu cầu nhiều mô hình hơn để triệt tiêu phương sai.
- Hiệu ứng chiều rộng vô hạn trở nên đáng kể hơn, làm sai lệch dự đoán của bộ dự báo NTK so với mạng thực tế.

2. Giới hạn phần cứng:

Việc tăng chiều rộng lớp ẩn để tiệm cận chế độ NTK lý tưởng bị hạn chế bởi tài nguyên tính toán:

- **Bộ nhớ GPU:** Mỗi mô hình với $n_h = 50,000$ tiêu tốn đáng kể bộ nhớ. Việc tăng n_h lên 100,000 hoặc cao hơn vượt quá khả năng của GPU hiện có.
- **Thời gian huấn luyện:** Với ensemble gồm 300 mô hình, thời gian huấn luyện tổng cộng đã lên đến nhiều giờ. Việc tăng n_h sẽ làm tăng thời gian này theo bậc hai.
- **Tính toán ma trận NTK:** Với bài toán cộng, kích thước ma trận kernel tăng nhanh theo số bit, làm tăng chi phí tính toán nghịch đảo ma trận.

5.6 Kết luận từ thực nghiệm

Kết quả thực nghiệm cho thấy khoảng cách giữa lý thuyết và thực tiễn trong việc triển khai khung NTK. Mặc dù lý thuyết đảm bảo độ chính xác tuyệt đối với điều kiện $n_h \rightarrow \infty$, việc đạt được điều kiện này trong thực tế đòi hỏi:

- Phần cứng tính toán mạnh hơn (GPU với bộ nhớ lớn hơn, hoặc cụm GPU phân tán).
- Các kỹ thuật xấp xỉ kernel hiệu quả để giảm chi phí tính toán.
- Nghiên cứu thêm về ảnh hưởng của chiều rộng vô hạn đến độ chính xác.

Đây là một hướng phát triển quan trọng cho các nghiên cứu tương lai về việc đưa khung lý thuyết NTK vào ứng dụng thực tiễn.

5.7 Ý nghĩa thực tiễn

Kết quả thực nghiệm xác nhận hai điều quan trọng:

1. **Tính đúng đắn của lý thuyết:** Ngưỡng lý thuyết luôn là chặn trên hợp lệ cho số mô hình cần thiết.
2. **Khả năng thực tiễn:** Trong ứng dụng thực tế, có thể sử dụng ít mô hình hơn so với ngưỡng lý thuyết mà vẫn đạt độ chính xác cao (như đã thấy trong bài toán hoán vị).

Chương 6

PHÂN TÍCH PHÊ BÌNH

Hạn chế của phương pháp: Mặc dù đạt được các kết quả lý thuyết quan trọng, phương pháp vẫn tồn tại một số hạn chế cần được nhận thức rõ:

- **Tính trực giao của dữ liệu:** Phương pháp phụ thuộc mạnh vào giả định rằng dữ liệu huấn luyện được trực giao hóa và cấu trúc hóa theo dạng phân vùng khối. Trong các tập dữ liệu tự nhiên, tính chất trực giao này khó đạt được một cách tự động.
- **Bộ nhớ hữu hạn:** Mô hình bị giới hạn bởi kích thước đầu vào cố định. Khi thay đổi kích thước bit - ví dụ từ 10 bit lên 20 bit - mạng cần được huấn luyện lại hoàn toàn do kiến trúc không tự động mở rộng. Điều này hạn chế tính linh hoạt trong ứng dụng thực tế.
- **Khả năng tổng quát hóa độ dài:** Phương pháp chưa đạt được khả năng tổng quát hóa độ dài - khả năng tổng quát hóa cho các chuỗi đầu vào dài hơn so với dữ liệu huấn luyện. Đây là một hạn chế so với các kiến trúc RNN hay Transformer hiện đại [13], vốn có khả năng xử lý ngoài phân phối về độ dài một cách tự nhiên hơn.

Chương 7

KẾT LUẬN

7.1 Tổng kết nghiên cứu

Nghiên cứu này đã giải quyết thành công câu hỏi cốt lõi về khả năng học và thực thi chính xác tuyệt đối các thuật toán nhị phân của mạng nơ-ron hai lớp trong giới hạn chiều rộng vô hạn ($n_h \rightarrow \infty$). Thông qua việc kết hợp kỹ thuật khớp mẫu với phân tích Neural Tangent Kernel (NTK), nghiên cứu đã đưa ra các chứng minh lý thuyết vững chắc được xác nhận bởi thực nghiệm.

Các kết quả chính đạt được:

1. **Chứng minh *Khả năng học chính xác*:** Nghiên cứu đã chứng minh khả năng học được chính xác cho bốn nhóm tác vụ cơ bản:
 - Phép hoán vị
 - Phép cộng nhị phân
 - Phép nhân nhị phân
 - Chỉ thị SBN
2. **Hiệu quả dữ liệu vượt trội:** Phương pháp chỉ yêu cầu số lượng mẫu huấn luyện theo bậc logarit so với kích thước không gian đầu vào ($O(\log 2^l) = O(l)$ thay vì $O(2^l)$), giúp việc huấn luyện khả thi cho các bài toán với số bit lớn.
3. **Vai trò của lý thuyết NTK:** Khung toán học NTK cho phép dự báo chính xác hành vi của mạng mà không cần huấn luyện thực tế, cung cấp công cụ phân tích mạnh mẽ cho việc thiết kế và đánh giá hệ thống.

7.2 Ý nghĩa của nghiên cứu

Tính phổ quát của kết quả:

Việc thực thi thành công chỉ thị SBN - vốn được chứng minh là có tính đầy đủ Turing - có ý nghĩa sâu sắc: về nguyên tắc, mạng nơ-ron có khả năng thực thi mọi hàm số tính toán được. Điều này thiết lập cầu nối lý thuyết giữa khả năng biểu diễn của mạng nơ-ron và lý thuyết tính toán cổ điển.

Đóng góp lý thuyết:

Đây là chứng minh dựa trên NTK đầu tiên cho việc thực thi thuật toán chính xác. Khác với các nghiên cứu trước đó chỉ tập trung vào xấp xỉ các hàm trơn, nghiên cứu này chứng minh rằng mạng nơ-ron có thể học các hàm rời rạc với độ chính xác tuyệt đối thông qua cơ chế làm tròn.

7.3 Hướng phát triển tương lai

Dựa trên các kết quả và hạn chế đã phân tích, một số hướng nghiên cứu tiềm năng được đề xuất:

1. **Dữ liệu có tương quan:** Nghiên cứu khả năng học được chính xác khi dữ liệu huấn luyện có tương quan thay vì tuân theo giả định trực giao nghiêm ngặt.
2. **Kiến trúc có độ dài thay đổi:** Mở rộng khung lý thuyết NTK sang các kiến trúc có khả năng xử lý đầu vào với độ dài thay đổi như Transformer, RNN, hoặc GNN trên đồ thị có bậc bị chặn.
3. **Học chỉ thị từ dữ liệu:** Khám phá khả năng mạng tự suy diễn ra các chỉ thị nguyên thủy từ các cặp đầu vào-đầu ra thực tế, thay vì được cung cấp sẵn các mẫu như trong phương pháp hiện tại.

Kết luận: Nghiên cứu này đã chứng minh rằng ranh giới giữa tính toán logic-thuật toán và học máy không phải là không thể vượt qua. Việc mạng nơ-ron có thể học thực thi chính xác các phép toán cơ bản - và thậm chí là các chỉ thị có tính đầy đủ Turing - mở ra triển vọng về một thế hệ hệ thống AI mới: kết hợp khả năng học từ dữ liệu của mạng nơ-ron với độ tin cậy và tính chính xác của tính toán thuật toán truyền thống.

Tài liệu tham khảo

- [1] Artur Back de Luca, George Giapitzakis, and Kimon Fountoulakis, “Learning to Add, Multiply, and Execute Algorithmic Instructions Exactly with Neural Networks”, arXiv:2502.16763, 2025.
- [2] Arthur Jacot, Franck Gabriel, and Clément Hongler, “Neural Tangent Kernel: Convergence and Generalization in Neural Networks”, *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.
- [3] Jaehoon Lee, Yasaman Bahri, Roman Novak, Samuel S. Schoenholz, Jeffrey Pennington, and Jascha Sohl-Dickstein, “Deep Neural Networks as Gaussian Processes”, *International Conference on Learning Representations (ICLR)*, 2018.
- [4] Greg Yang, “Tensor Programs I: Wide Feedforward or Recurrent Neural Networks of Any Architecture are Gaussian Processes”, arXiv:1910.12478, 2019.
- [5] Lénaïc Chizat and Francis Bach, “On the Global Convergence of Gradient Descent for Over-Parameterized Models using Optimal Transport”, *Advances in Neural Information Processing Systems (NeurIPS)*, 2018. (Bản mở rộng: arXiv:1805.09545, 2019.)
- [6] Ali Rahimi and Benjamin Recht, “Random Features for Large-Scale Kernel Machines”, *Advances in Neural Information Processing Systems (NeurIPS)*, 2008.
- [7] Christopher K. I. Williams, “Computing with Infinite Networks”, *Advances in Neural Information Processing Systems (NeurIPS)*, 1997.
- [8] Youngmin Cho and Lawrence K. Saul, “Kernel Methods for Deep Learning”, *Advances in Neural Information Processing Systems (NeurIPS)*, 2009.
- [9] Bernhard Schölkopf and Alexander J. Smola, *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*, MIT Press, 2002.
- [10] Sanjeev Arora, Simon S. Du, Wei Hu, Zhiyuan Li, Ruslan Salakhutdinov, and Ruosong Wang, “On Exact Computation with an Infinitely Wide Neural Net”, *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [11] Simon S. Du, Jason D. Lee, Haochuan Li, Liwei Wang, and Igor S. Dhillon, “Gradient Descent Finds Global Minima of Deep Neural Networks”, *International Conference on Machine Learning (ICML)*, 2019.
- [12] Song Mei, Andrea Montanari, and Phan-Minh Nguyen, “A Mean Field View of the Landscape of Two-Layer Neural Networks”, *Proceedings of the National Academy of Sciences (PNAS)*, 115(33):E7665–E7671, 2018.

- [13] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin, “Attention Is All You Need”, *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [14] Wojciech Zaremba and Ilya Sutskever, “Learning to Execute”, arXiv:1410.4615, 2014.
- [15] Alex Graves, Greg Wayne, and Ivo Danihelka, “Neural Turing Machines”, arXiv:1410.5401, 2014.
- [16] Alex Graves, Greg Wayne, Malcolm Reynolds, Tim Harley, Ivo Danihelka, Agnieszka Grabska-Barwińska, Sergio Gómez Colmenarejo, Edward Grefenstette, Tiago Ramalho, John Agapiou, Adrià Puigdomènech Badia, Karl Moritz Hermann, Yori Zwols, Georg Ostrovski, Adam Cain, Helen King, Christopher Summerfield, Phil Blunsom, Koray Kavukcuoglu, and Demis Hassabis, “Hybrid Computing Using a Neural Network with Dynamic External Memory”, *Nature*, 538:471–476, 2016.
- [17] Łukasz Kaiser and Ilya Sutskever, “Neural GPUs Learn Algorithms”, *International Conference on Learning Representations (ICLR)*, 2016 (arXiv:1511.08228, 2015).
- [18] Scott Reed and Nando de Freitas, “Neural Programmer-Interpreters”, *International Conference on Learning Representations (ICLR)*, 2016.
- [19] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”, *NAACL-HLT*, 2019 (arXiv:1810.04805, 2018).
- [20] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al., “Language Models are Few-Shot Learners”, *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [21] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Łukasz Kaiser, Matthias Plappert, Fotios Petroni, and others, “Training Verifiers to Solve Math Word Problems”, arXiv:2110.14168, 2021. (Dataset GSM8K.)
- [22] Chulhee Yun, Srinadh Bhojanapalli, Ankit Singh Rawat, Sashank J. Reddi, and Sanjiv Kumar, “Are Transformers Universal Approximators of Sequence-to-Sequence Functions?”, *International Conference on Learning Representations (ICLR)*, 2020.
- [23] Jorge Pérez, Pablo Barceló, and Javier María Tur, “The Computational Power of Transformers”, *International Conference on Learning Representations (ICLR)*, 2021.
- [24] John E. Hopcroft and Jeffrey D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, Addison-Wesley, 1979.
- [25] Marvin Minsky, *Computation: Finite and Infinite Machines*, Prentice-Hall, 1967.
- [26] Hava T. Siegelmann and Eduardo D. Sontag, “On the Computational Power of Neural Nets”, *Journal of Computer and System Sciences*, 50(1):132–150, 1995.
- [27] Alan M. Turing, “On Computable Numbers, with an Application to the Entscheidungsproblem”, *Proceedings of the London Mathematical Society*, 42(2):230–265, 1936.

- [28] Stephen A. Cook, “The Complexity of Theorem-Proving Procedures”, *Proceedings of the Third Annual ACM Symposium on Theory of Computing (STOC)*, 1971.